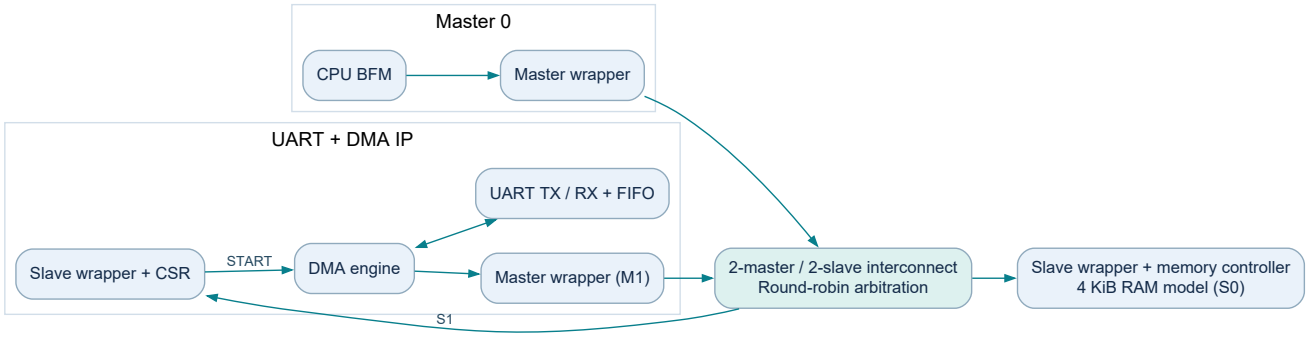
HW1：UART + DMA AHB / AXI4 介面設計

這次作業選 UART + DMA 當作例子，分別用 AHB 和 AXI4 連接。內容包括 Master / Slave wrapper 的狀態圖、訊息循序圖，以及 RTL 和模擬結果。



元件	在系統裡的角色
CPU BFM（M0）	在測試中代替 CPU 發出讀寫要求，用來設定 IP、檢查記憶體。
IP 內部 DMA（M1）	主動讀寫記憶體，一次搬一筆或用 burst 搬四筆資料。
Memory Controller（S0）	收到讀寫要求後，幫忙讀寫 4 KiB 的 RAM。
IP 控制介面（S1）	讓 CPU 讀寫控制暫存器，設定 UART 和 DMA。

可以這樣看：先看這頁有哪些元件，再看第 2 頁的設定。第 3-7 頁是第一題，第 8-11 頁是第二題，後面放模擬結果、波形和功能範圍。

README 裡有檔案位置和執行方式；圖稿、程式與模擬紀錄也都有附上。

設計範圍與暫存器

AHB 和 AXI4 兩個版本都接同一份 uart_dma 核心。CPU 和 DMA 共用匯流排，一次處理一筆交易；這筆結束後，再用 round robin 決定接下來換誰。

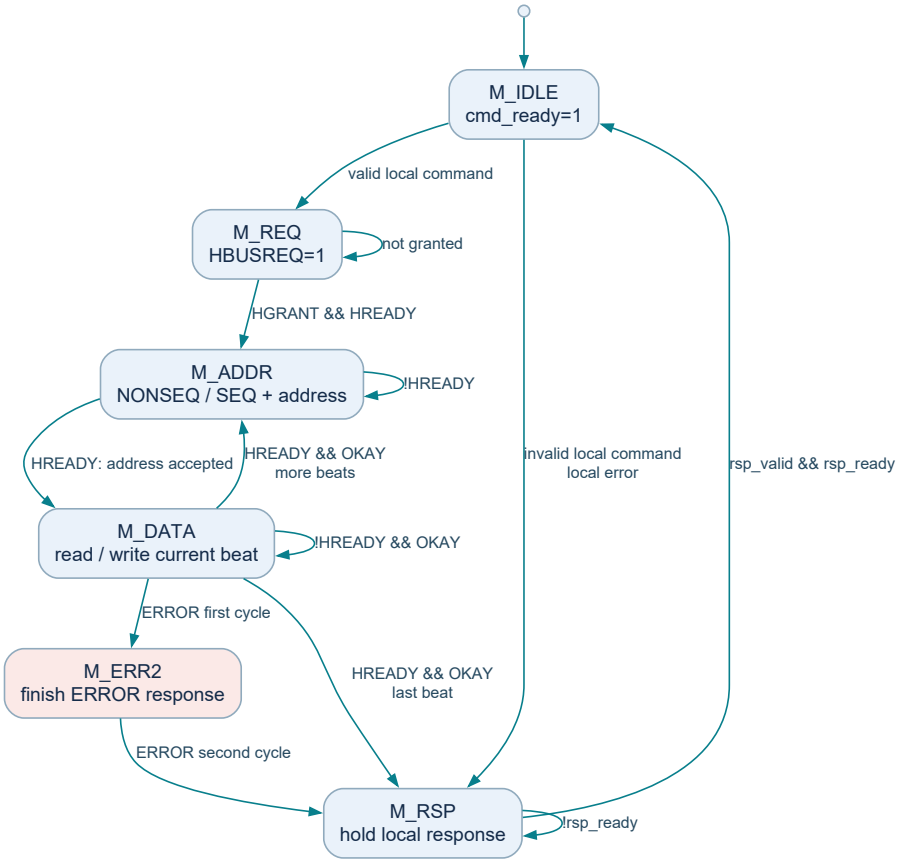
項目	本作業的選擇
AHB	採用 AMBA 2 AHB，支援 OKAY / ERROR、request / grant，以及 SINGLE、INCR4 傳輸。
AXI	採用 AXI4 memory-mapped，支援 INCR 的 1 或 4 beats；AW 和 W 各自握手。
資料與位址	資料寬度為 32-bit，位址要 4-byte 對齊，每傳一筆位址加 4。每個 Master 都要等前一筆交易完成，才能接下一筆。
記憶體 / IP	RAM：0x00000000-0x00000FFF；IP：0x40000000-0x400000FF。
UART	使用 8N1，模擬時每個 bit 持續 8 個時脈。每個 32-bit word 放一個 UART byte，使用 bits[7:0]。

Offset	名稱 / 使用方式
0x00 / 0x04	TXDATA：寫入一個要送出的 byte；RXDATA：讀出最早收到的 byte，並從 FIFO 移除。
0x08	STATUS：bit0 busy、bit1 done、bit2 error、bit3 TX busy、bits[7:4] RX FIFO count。
0x0C / 0x10	DMA_SRC / DMA_DST：記憶體來源 / 目的位址。
0x14	DMA_LEN：1 或 4，單位是 word。
0x18	DMA_CTRL：bit0 START；bit1 方向。寫 1：memory→UART；寫 3：UART→memory。

例如 SRC=0x100、LEN=4，四個 word 的低 byte 分別是 48、57、31、21（十六進位），UART 就會依序送出「HW1!」。所有 FSM 都在時脈上升緣處理，resetn=0 時回到 IDLE。還沒達到圖上的轉換條件時，就留在原本的狀態。

第一題 A：AHB Master wrapper

CPU 和 DMA 各有一份 Master wrapper，負責把內部命令換成 AHB 訊號。兩邊都想傳資料時，由仲裁器決定誰先使用匯流排。

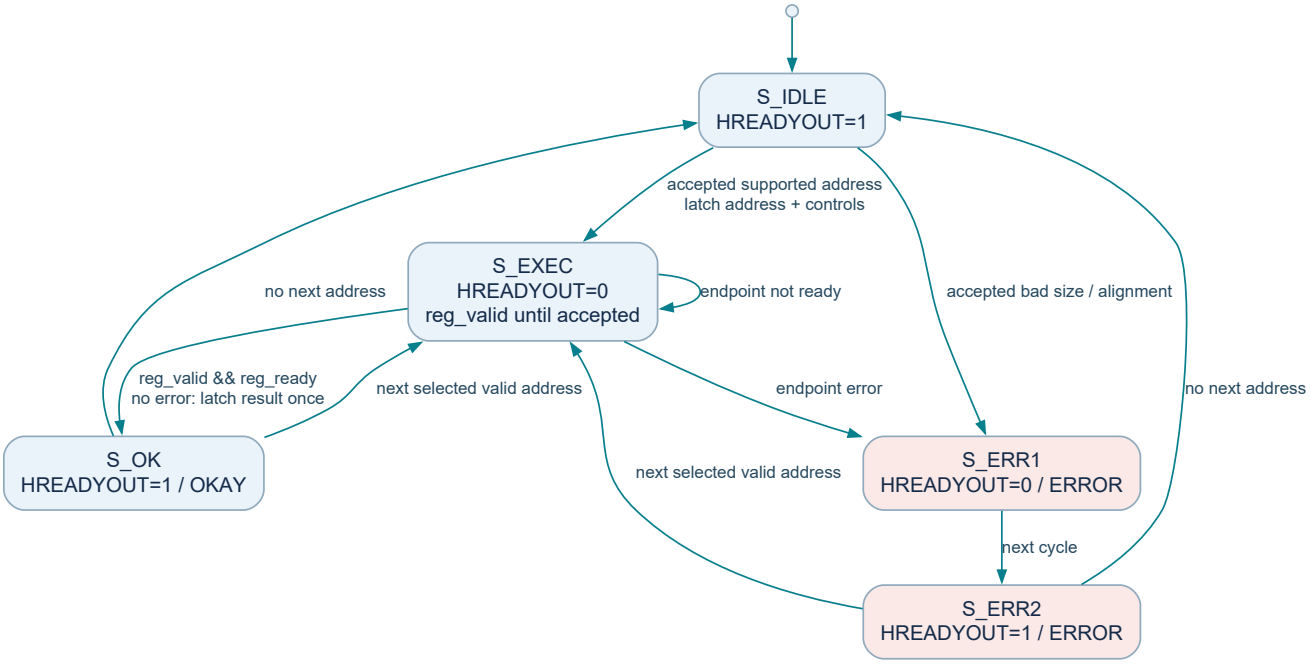


狀態	白話說明
M_IDLE / M_REQ	先收下命令，再用 HBUSREQ 申請使用匯流排。等 HGRANT 和 HREADY 都為 1，才開始傳輸。
M_ADDR	送出位址和控制訊號。第一筆設 HTRANS=NONSEQ，後面幾筆用 SEQ。
M_DATA	等這筆資料傳完。讀取時收下 HRDATA；寫入時保持 HWDATA。這筆成功完成後，才換下一筆。
M_ERR2 / M_RSP	若發生錯誤，先等兩拍 ERROR 回覆結束，再把結果交回 CPU 或 DMA。正常完成也會進入回覆狀態。

這裡把位址和資料分開處理，讓狀態比較好看懂：還有下一筆時，資料階段送 BUSY；最後一筆則送 IDLE。INCR4 仍然會傳四筆有效資料。等待時，正在使用的訊號要保持不變。

第一題 B：AHB Slave wrapper

Slave wrapper 被選中後，先記住這筆的位址和控制訊號，再請內部 RAM 或 IP 暫存器處理，最後回覆結果。

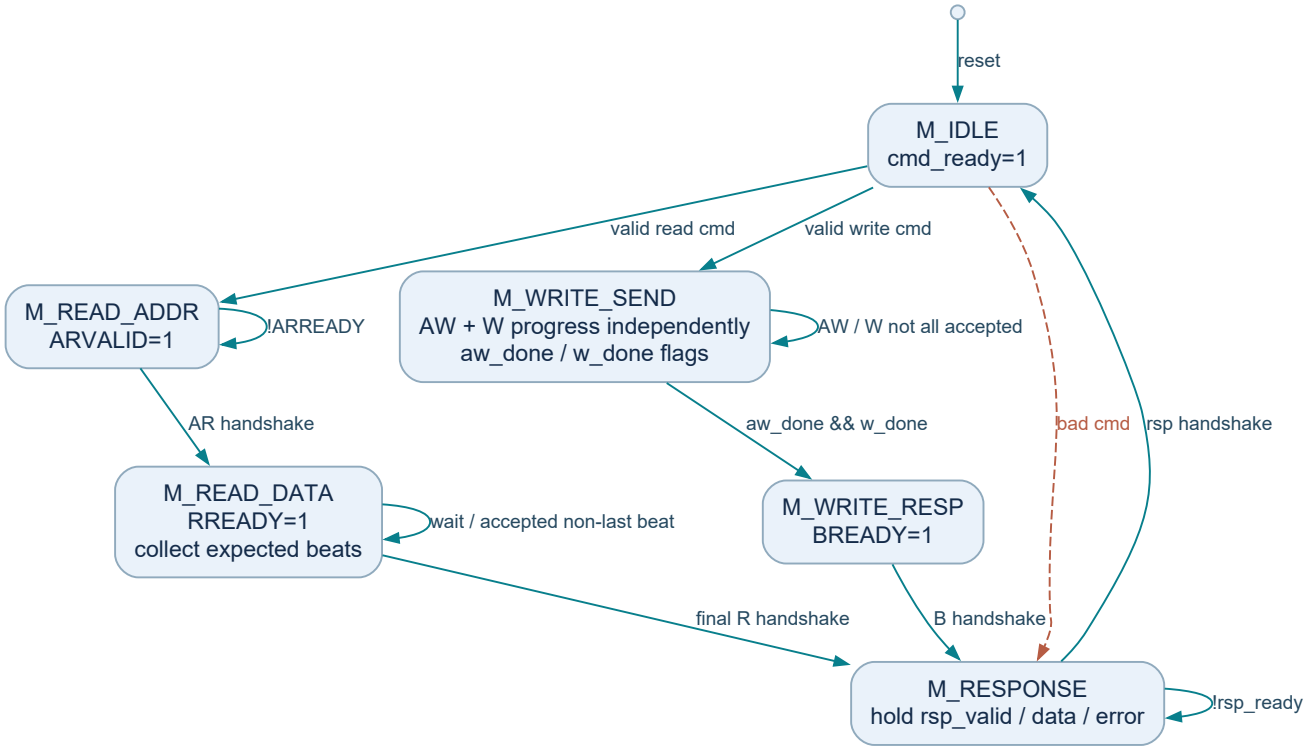


訊號 / 狀態	要注意的事情
有效要求	HSEL、HREADY、HTRANS[1] 都為 1 時，才收下這筆位址。IDLE 和 BUSY 都不算有效讀寫。
S_EXEC	等內部 RAM 或暫存器準備好。接收成功後先記住結果，再回覆匯流排；同一筆只處理一次。
S_OK	用 HREADYOUT=1、HRESP=OKAY 表示成功；讀取時也送回剛才記住的 HRDATA。
S_ERR1 / S_ERR2	錯誤要分兩拍回覆：第一拍 HREADYOUT=0，第二拍為 1，兩拍的 HRESP 都是 ERROR。

每收到新位址，都要檢查 SIZE 和對齊方式；有問題就進 S_ERR1。從 S_OK、S_ERR2 接下一筆時也一樣。內部模組接收後就停止重送，避免寫入兩次，或把 RX FIFO 多讀一筆。

第一題 C：AXI4 Master wrapper

AXI 把讀寫分成五個通道：讀取用 AR、R，寫入用 AW、W、B。到了時脈上升緣，VALID 和 READY 都為 1，這個通道上的一筆資料才算傳成功，也就是完成一次握手。

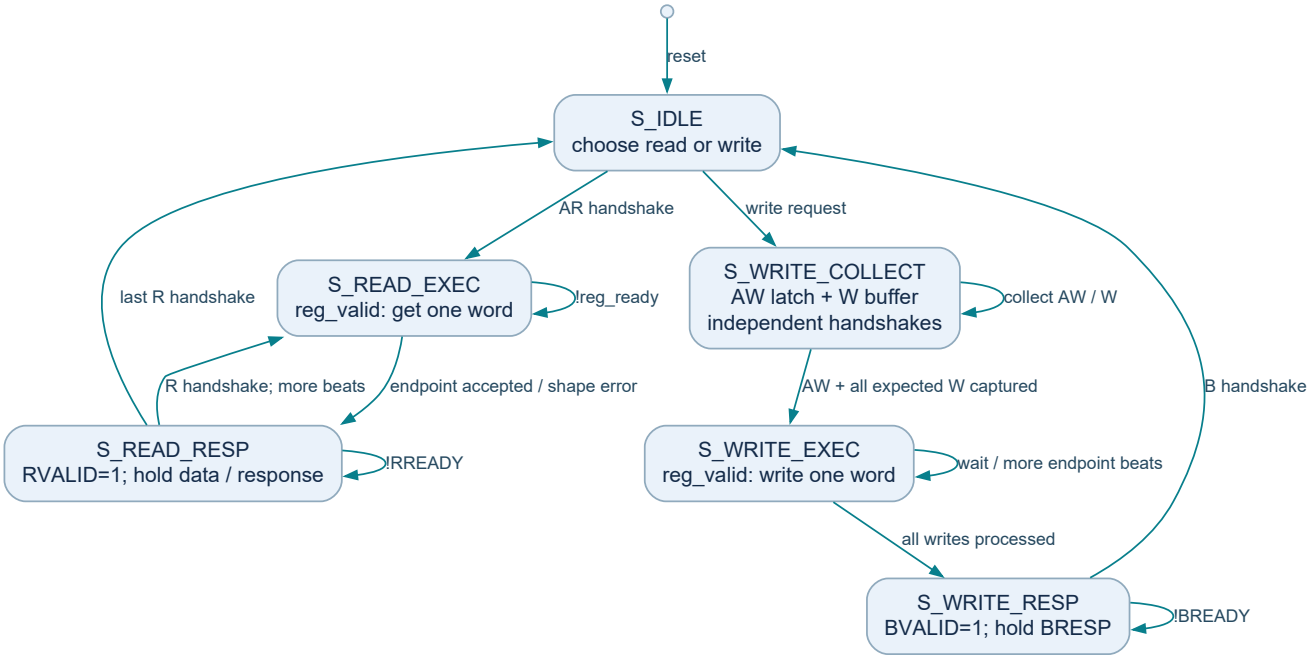


工作	狀態圖的讀法
讀取	先用 AR 送位址，再從 R 一筆一筆收資料。R 握手成功才增加計數，最後一筆要有 RLAST。
寫入	AW 負責送位址，W 負責送資料，兩邊分開記錄進度，可以在不同拍完成。最後一筆 W 要有 WLAST。
寫回覆	位址和資料都送完後，還要等 B 回覆。B 握手成功，這次寫入才算完成。
四筆 burst	設 AxLEN=3、AxSIZE=2（每筆 4 bytes）、AxBURST=INCR。若跨過 4 KiB 邊界或命令格式有誤，就直接回報錯誤，不送上匯流排。

VALID 已經是 1、READY 還是 0 時，就要等對方準備好，期間保持 VALID 和資料不變。讀寫有錯誤時，用 rsp_error 回報，再由 DMA 記到 STATUS。

第一題 D：AXI4 Slave wrapper

Slave 收到 AXI 要求後，會一筆一筆交給內部 RAM 或暫存器處理。讀回的資料和寫入回覆都要先保留，等對方用 READY 表示接收。

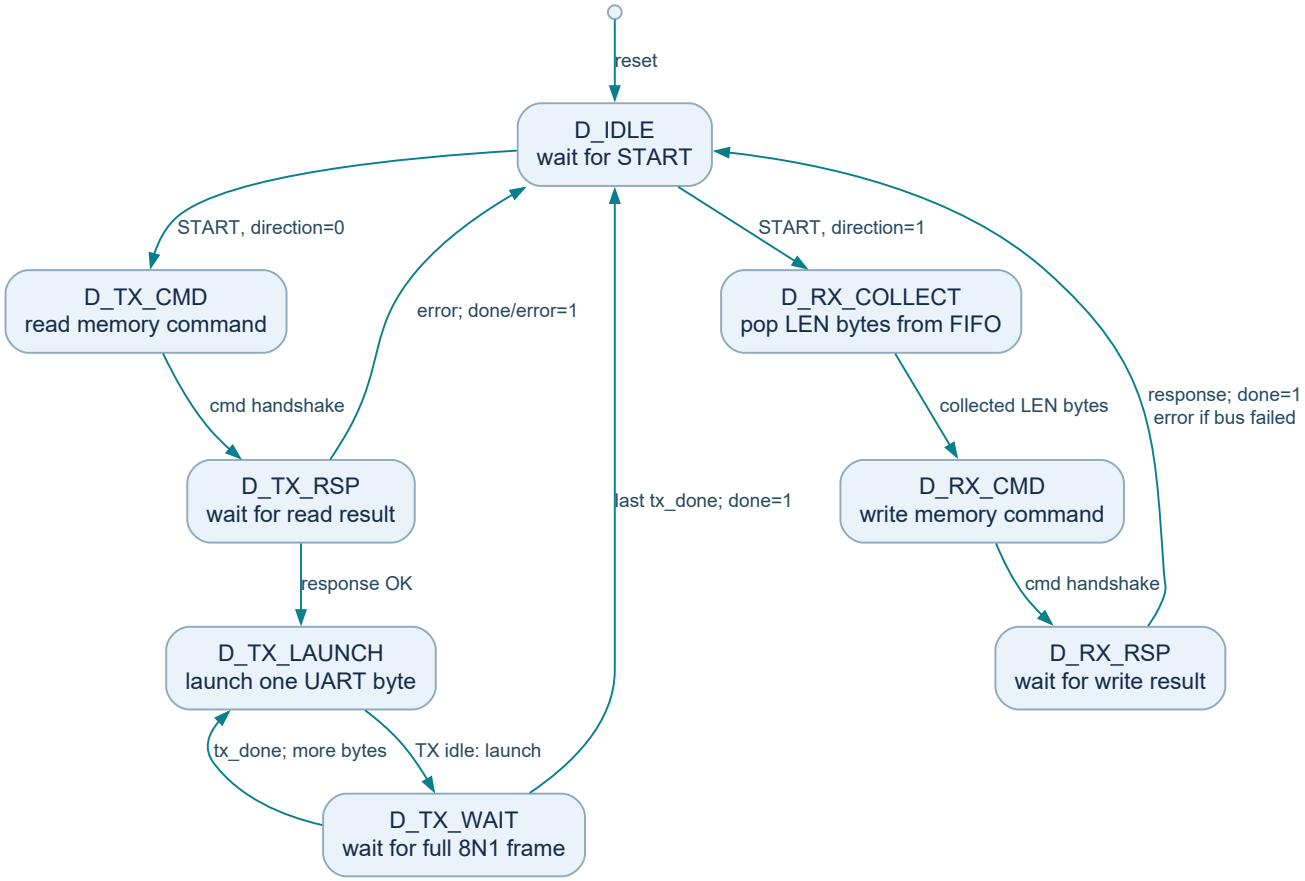


工作	狀態圖的讀法
AR / R	先記住讀取位址，拿到內部資料後再用 R 送出。對方還沒收下這筆，就先不處理下一筆。
AW / W	位址和資料分開接收、分開保存。等這次寫入的資訊收齊，才開始寫進內部模組。
B	預期的 W 資料都收到、內部處理也結束後，才送 B 回覆。BREADY=0 時，繼續保持 BVALID 和 BRESP。
錯誤	位址找不到對應模組時回報解碼錯誤；內部處理失敗則回 SLVERR。burst 發生錯誤時，仍要把這筆交易該有的資料和回覆流程走完。

這裡一次處理一筆交易，方便安排資料緩衝和連接。AW、W 怎麼接收，以及怎麼保留目前的使用者，都寫在 docs/axi_notes.md 和對應 RTL 裡。

第一題補充：DMA 與多 Master

下面畫的是 IP 裡的 DMA，會接到前面說明的 wrapper。CPU 設定好並啟動後，DMA 就能自己讀寫記憶體，因此它也是一個 Master。



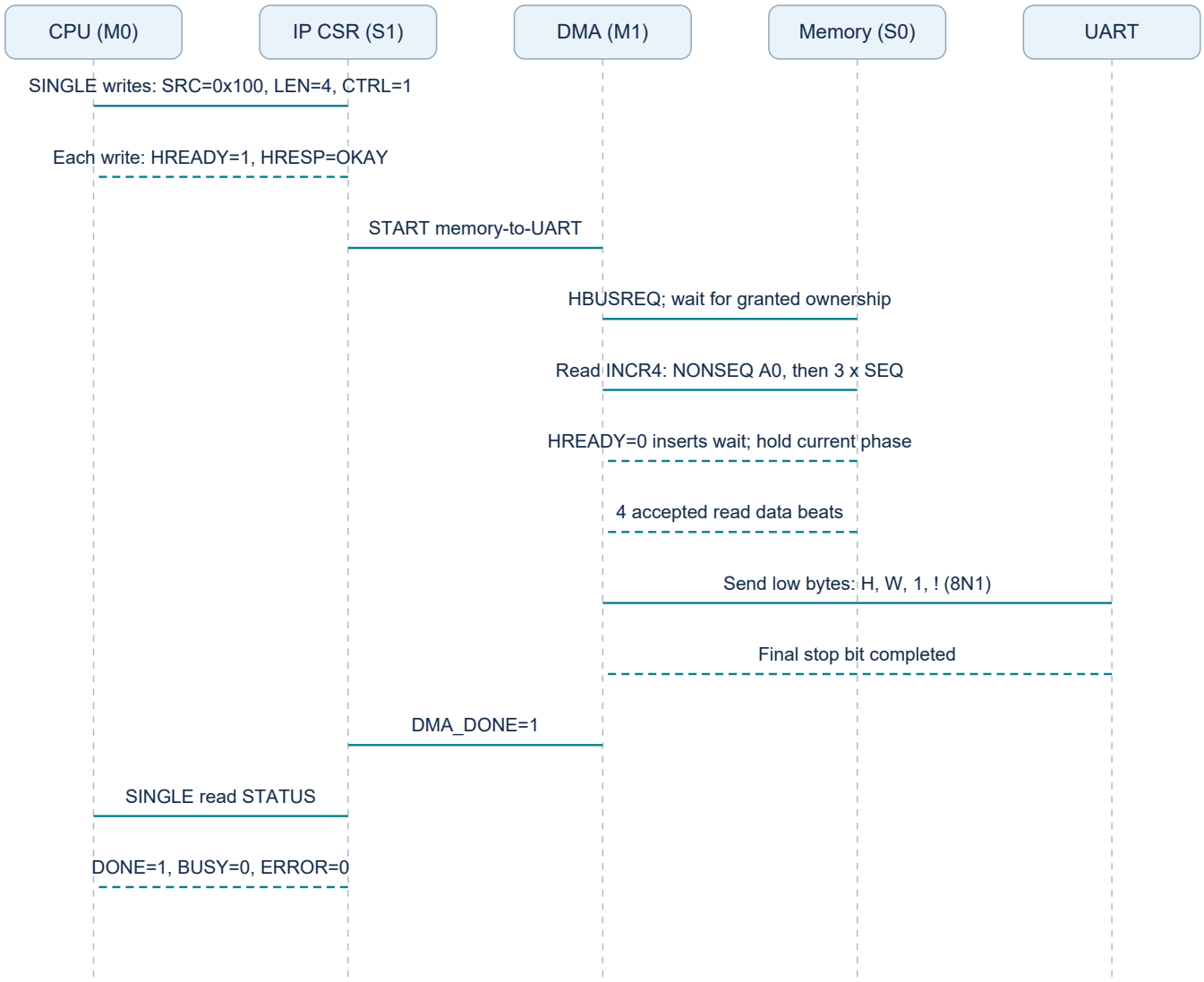
TX：先把這次要傳的資料讀回來，再交給 UART 一個字元接一個字元送出。最後一個 stop bit 送完，才設 done。
RX：從 RX FIFO 收滿 LEN 個 byte，放進 1 或 4 個 word 後寫入記憶體；收到寫入回覆才設 done。

誰先用匯流排：兩個 Master 同時提出要求時，優先換上次沒用到的那個。每筆交易做完才換人。AHB 用 request / grant 管理；AXI 則等最後的 R 或 B 回覆完成，再讓下一個 Master 使用。

仲裁圖放在 diagrams/07_arbiter.dot 和 .svg。UART FIFO 可以放 8 bytes；RX 還沒收滿資料時，就繼續等。

第二題 A：AHB 的傳送流程

這個例子先由 CPU 設定 IP，再讓 DMA 從記憶體讀四個 word，交給 UART 送出「HW1!」。圖上的箭頭表示一次溝通或一組傳輸，可能需要好幾個時脈。

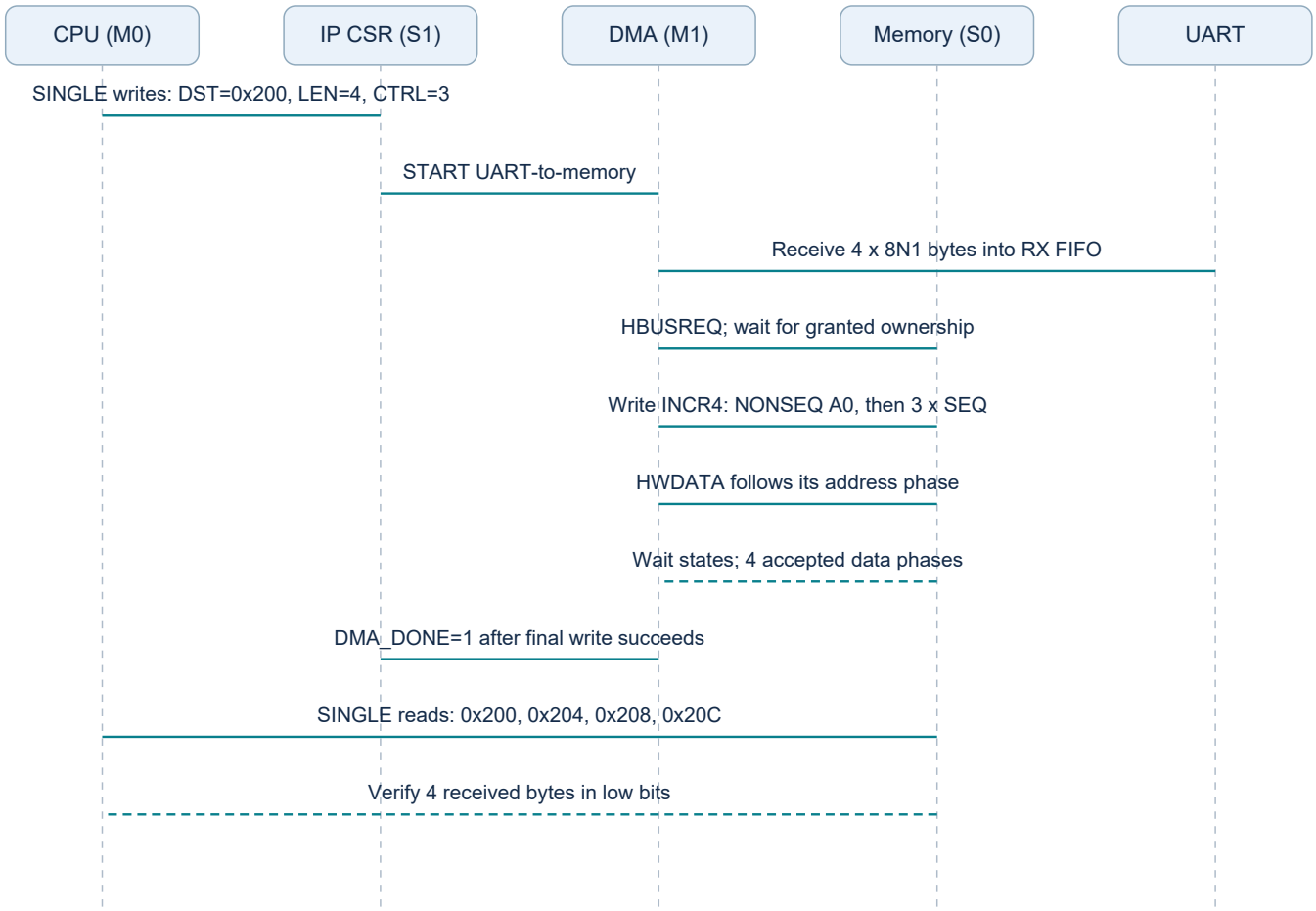


CPU 設定 IP 時，資料走 M0→S1；換成 DMA 讀記憶體時，走的是 M1→S0。所以同一個 IP 裡，同時有接收設定的 Slave 介面，以及主動搬資料的 Master 介面。

HBUSREQ / HGRANT 由仲裁器處理，圖中把這段註記在 DMA 提出要求的位置，方便閱讀。HREADY=0 表示還要等，這時還不能把資料算成已傳完。

第二題 B：AHB 的接收流程

這次換成接收：CPU 啟動 DMA 後，等 UART 收到四個 byte，再由 DMA 用 INCR4 寫到記憶體的 0x200、0x204、0x208、0x20C。

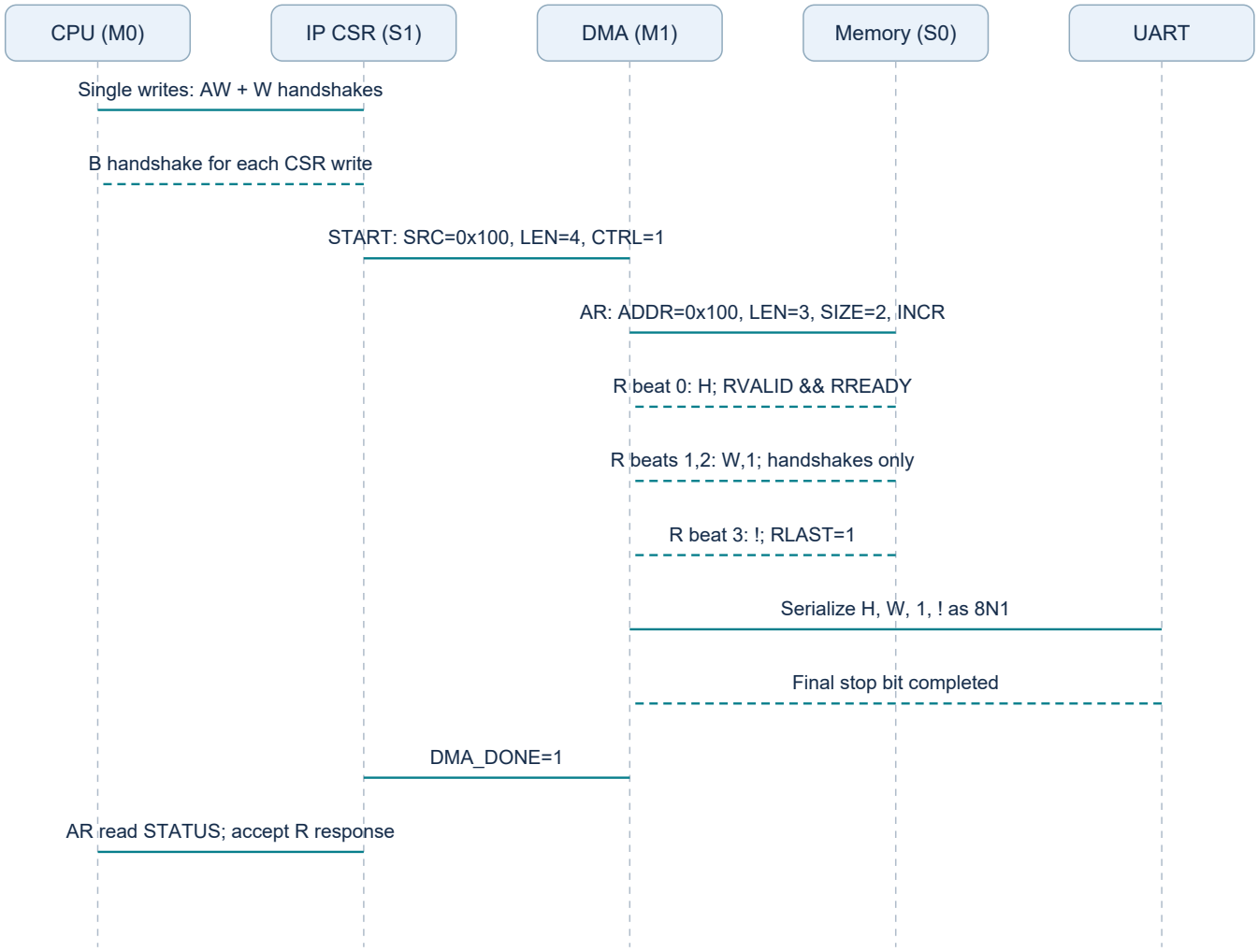


四筆寫入中，第一筆用 NONSEQ，後面三筆用 SEQ。位址和資料在不同階段傳送，所以 HWDATA 要對上前面已經收下的那筆位址。

最後一筆資料成功寫完，DMA 才回報正常完成。如果 Slave 回 ERROR，done 和 error 都會設起來，因此 CPU 還要看 STATUS.error，確認這次有沒有成功。

第二題 C：AXI4 的傳送流程

一樣是送出「HW1!」，這次改用 AXI4。DMA 用 AR 送一次讀取位址，再從 R 收回四筆資料。

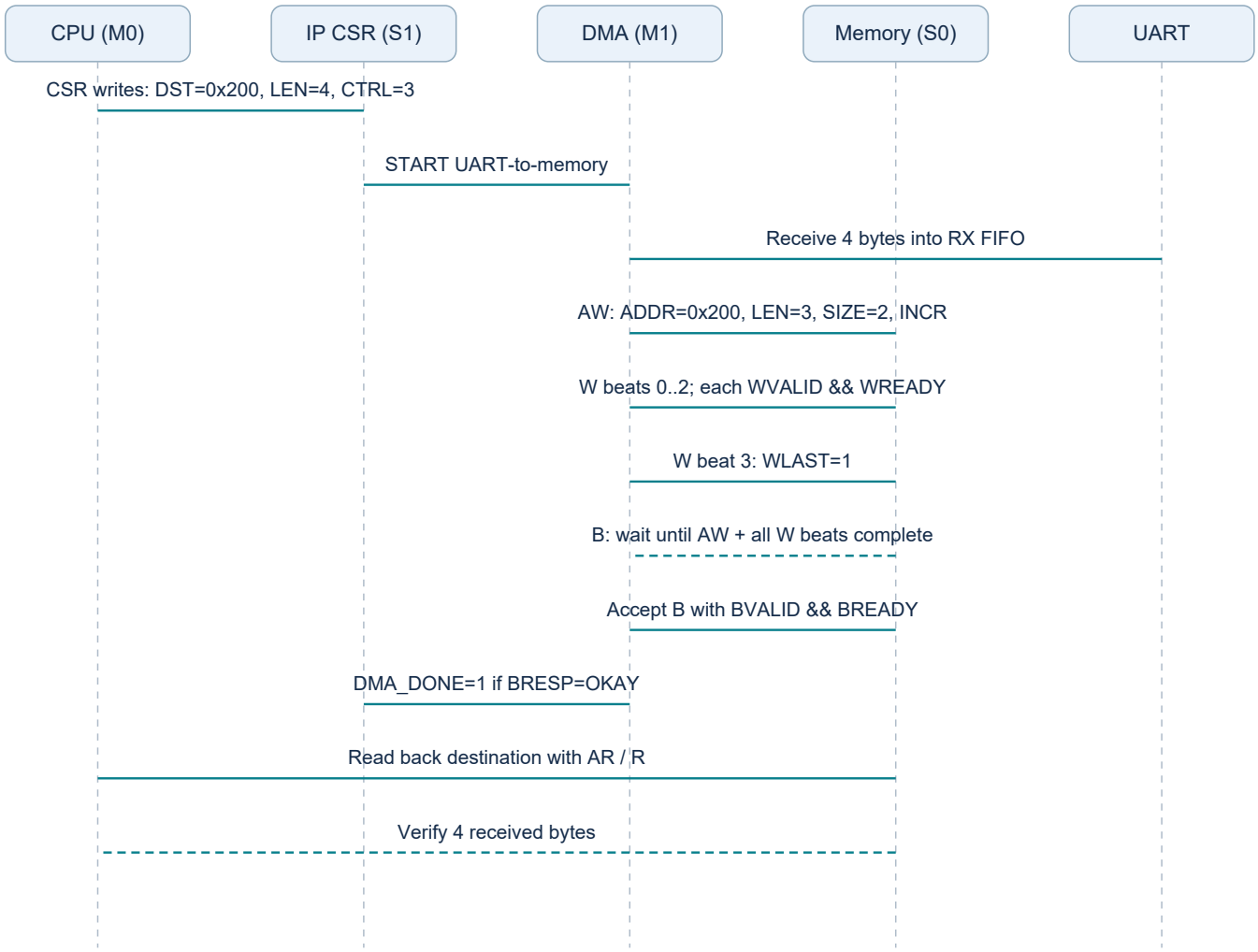


ARLEN=3 表示要讀 4 筆，ARSIZE=2 表示每筆 4 bytes。最後一筆，也就是 R beat 3，要帶 RLAST。RREADY=0 時，就保持這筆資料和回覆，等對方接收。

四筆資料都收回來後，再交給 UART 逐字元傳送。這時匯流排已經可以換人使用，UART 則繼續把剩下的 bits 送完。

第二題 D：AXI4 的接收流程

UART 收滿四個 byte 後，DMA 用 AW 送位址、W 送資料。下圖示範 AW 先完成；實際上兩個通道各自運作，也可能 W 先到，不能假設它們一定同拍完成。



最後一筆 W beat 3 要帶 WLAST。Slave 收齊位址和資料、完成內部寫入後，才送 B 回覆。DMA 收到成功的 B 回覆，才回報正常完成。

這裡每收到一個 byte，就放進一個 32-bit word。例如「RX4!」會分別寫成 0x00000052、0x00000058、0x00000034、0x00000021。

加分項：RTL 與自動檢查模擬

模擬時，SystemVerilog 測試程式會自動檢查資料、訊號和完成時機。有任何一項不符合預期，就會報錯並結束。下面是實際跑出的結果，完整紀錄放在 results/。

測試平台	實際執行結果
tb_uart_dma	PASS；941 個自動檢查；6 個 DMA 命令。
tb_ahb	PASS；44 次 INCR4；561 個等待週期；32 個競爭週期；5 次 ERROR。
tb_axi	PASS；332 個自動檢查；已測 AW 先到、W 先到，以及 R/B/W 等待的情況。

模擬器：Icarus Verilog version 12.0 (devel) (s20150603-1539-g2693dd32b)

驗證範圍	檢查內容
UART + DMA	一個 bit 一個 bit 比對「HW1!」的 8N1 傳送；也檢查接收順序、FIFO、DMA 完成時機、錯誤回覆和不合法的設定。
AHB 整合	兩個 Master、兩個 Slave；SINGLE / INCR4；等待狀態；錯誤處理；DMA 與 UART 收發。
AXI 整合	兩個 Master、兩個 Slave；單筆 / 四筆 INCR；五個通道的握手與等待；DMA 和 UART 的收發。

重新執行

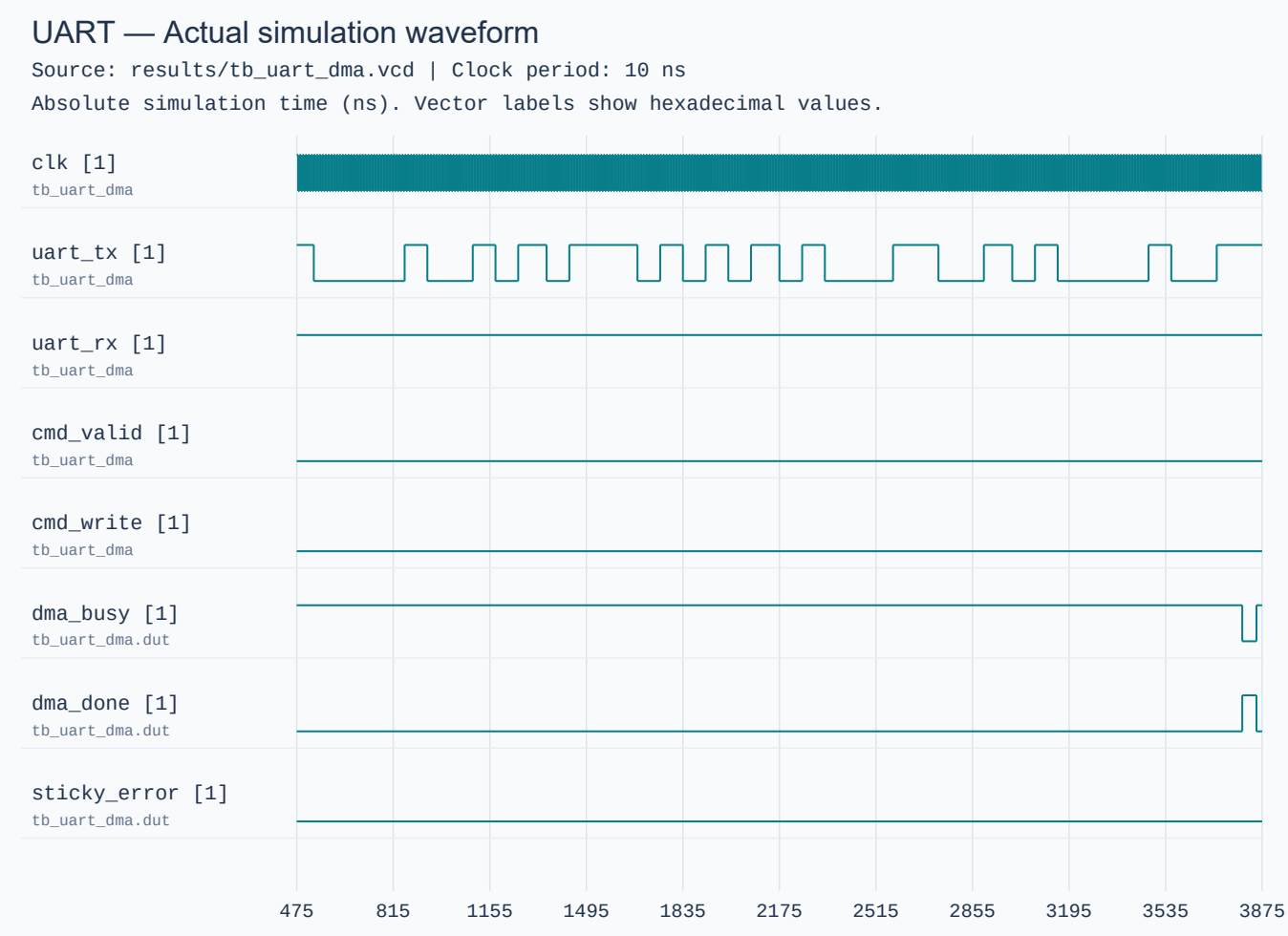
裝好 Icarus Verilog 和 Python 後，在 HW1 資料夾執行：
`python sim/run_all.py`
程式會重新編譯並跑完三組測試，產生 log、VCD、summary.json，以及用來核對原始碼的 SHA-256。

Windows 可以把模擬器裝在 C:/iverilog/bin，也可以用 IVERILOG、VVP 指定執行檔。重新跑 RTL 模擬時，準備好上述工具就可以了。

最後統一執行時間（UTC+8）：2026-10-15T02:00:04.946975+08:00

實際波形：UART 傳送

這張圖是從 tb_uart_dma 的 VCD 擷取的，包含第一次傳送四個字元的過程。uart_tx 沒傳資料時是 1；每個 frame 先送低電位 start，再送 8 個資料 bits 和 1 個 stop bit。測試程式也會逐 bit 比對內容，確認真的送對。

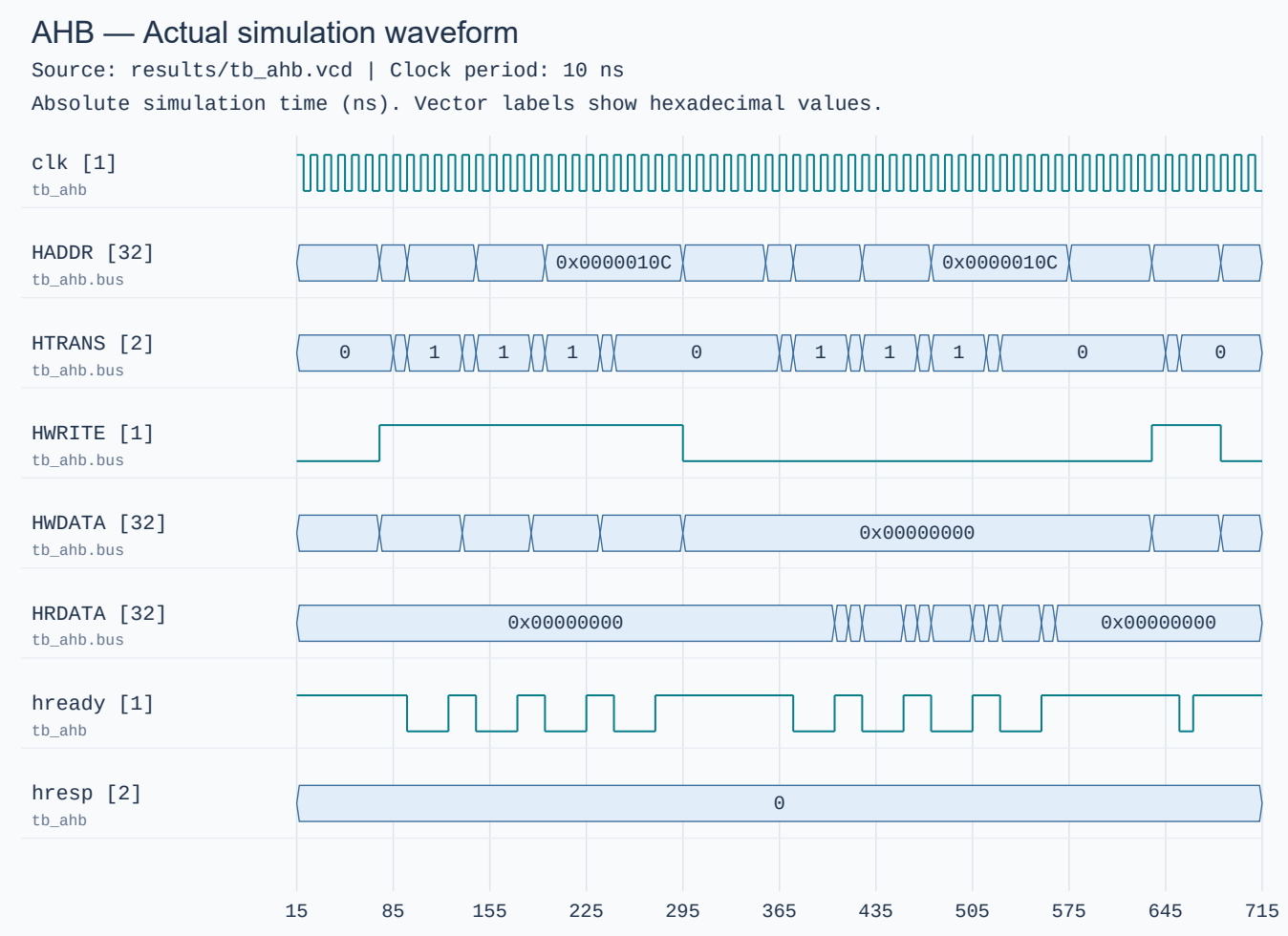


來源：results/tb_uart_dma.vcd。左邊也列出訊號所在的模組，方便分辨同名訊號。

想看完整過程或其他訊號，可以用 GTKWave 開啟 VCD。執行 sim/plot_vcd.py，也能從 VCD 重新畫出這張 SVG。

實際波形：AHB 四筆傳輸

這張圖擷取第一次 INCR4 附近的傳輸。HTRANS 的 0、1、2、3 分別是 IDLE、BUSY、NONSEQ、SEQ。HREADY=0 表示資料還沒傳完；圖裡也能看到中間插入的 BUSY，以及等待記憶體準備好的時間。

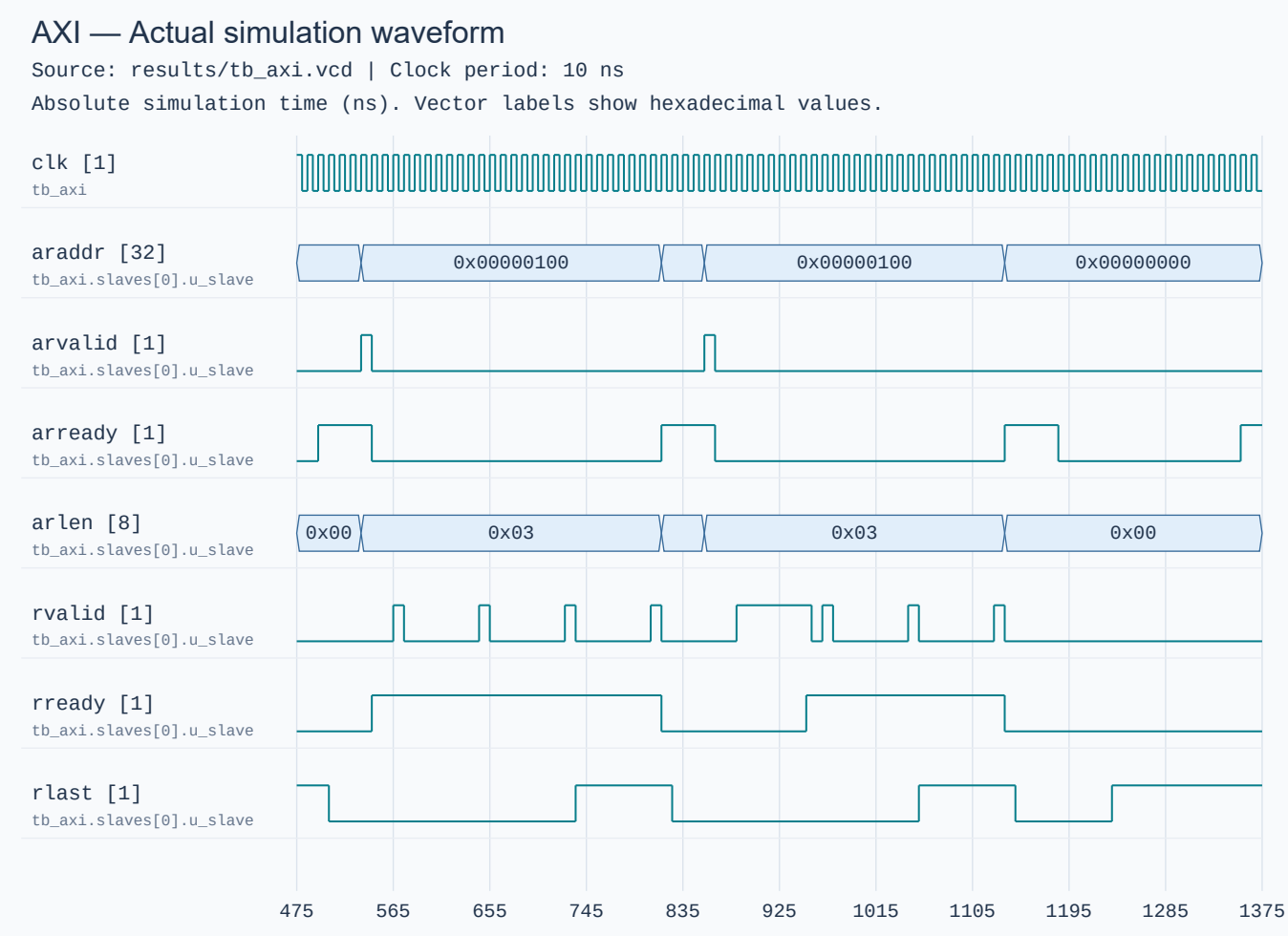


來源：results/tb_ahb.vcd。左邊也列出訊號所在的模組，方便分辨同名訊號。

想看完整過程或其他訊號，可以用 GTKWave 開啟 VCD。執行 sim/plot_vcd.py，也能從 VCD 重新畫出這張 SVG。

實際波形：AXI4 四筆讀取

這張圖擷取一次 ARLEN=3 的讀取。ARVALID 和 ARREADY 完成位址握手後，資料再透過 RVALID / RREADY 一筆一筆傳回，最後一筆會有 RLAST=1。圖中的時間使用實際模擬的 ns。



來源：results/tb_axi.vcd。左邊也列出訊號所在的模組，方便分辨同名訊號。

想看完整過程或其他訊號，可以用 GTKWave 開啟 VCD。執行 sim/plot_vcd.py，也能從 VCD 重新畫出這張 SVG。

提交內容、限制與參考資料

檔案 / 資料夾	用途
HW1_report.pdf	完整報告，直接開啟就能看。
README.md / docs/	執行方式、報告文字版，以及各模組的詳細說明。
diagrams/	可以修改的 DOT / Mermaid 原稿，以及直接能看的 SVG。
rtl/ / sim/	AHB、AXI、UART + DMA 的 RTL，以及自動檢查測試平台。
results/	實際測試紀錄、VCD 波形，以及能核對原始碼的 summary.json。

本次實作的範圍

這次主要用模擬檢查功能：CPU 由 BFM 代替，記憶體控制器用 RAM 模型，還沒有接外部 DRAM，也還沒上 FPGA 板子測試。AHB 這次沒有做 SPLIT / RETRY / LOCK；AXI 沒有做 WRAP、窄傳輸、非對齊、多筆 outstanding 或 out-of-order。

UART 使用固定參數的 8N1，沒有做 parity、流量控制、中斷，也不能在執行時更改 baud rate。RX 還沒收滿 LEN 個 byte 就會繼續等；如果資料進來時 FIFO 已經放滿，就會記錄錯誤。

參考資料

1. Arm，AMBA Specification Rev 2.0，IHI 0011A：AHB 傳輸、仲裁、burst 與回覆。
官方規格連結
2. Arm，AMBA AXI and ACE Protocol Specification，IHI 0022H：AXI4 握手、通道、burst 與回覆。
官方規格連結
3. OpenCores，UART16550：選擇 UART IP 時的功能參考。這份 RTL 另外實作，暫存器和功能不保證與 16550 相容。
專案頁面
4. Icarus Verilog：實際使用的 RTL 編譯 / 模擬工具。
官方文件

作業照片沒有指定協定版本和圖的張數，所以這裡先列出選用的版本、做到的功能和限制。如果課堂有另外規定，再照老師的要求調整。